

**Tuwaiq Academy**

Artificial Intelligence And Data Science Diploma

# **“Meimar” Real Estates Intelligent System**

Final Project Report

|                         |                         |
|-------------------------|-------------------------|
| <b>Student Name:</b>    | Mohammad Ahmad Al-Malke |
| <b>Instructor:</b>      | Banan Alnemri           |
| <b>Course:</b>          | Scalable Data Science   |
| <b>Submission Date:</b> | 13 / 08 / 2026          |

# Contents

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                 | <b>3</b>  |
| <b>2</b> | <b>Introduction</b>                             | <b>3</b>  |
| <b>3</b> | <b>Problem Statement</b>                        | <b>3</b>  |
| 3.1      | Background and Context . . . . .                | 3         |
| 3.2      | Why the Problem Matters . . . . .               | 4         |
| 3.3      | Problem Definition . . . . .                    | 4         |
| <b>4</b> | <b>Objectives</b>                               | <b>4</b>  |
| <b>5</b> | <b>Data</b>                                     | <b>5</b>  |
| 5.1      | Data Source . . . . .                           | 5         |
| 5.2      | Dataset Description . . . . .                   | 5         |
| 5.3      | Data Preprocessing . . . . .                    | 5         |
| 5.4      | Exploratory Data Analysis . . . . .             | 6         |
| 5.4.1    | The supply is one city . . . . .                | 7         |
| 5.4.2    | Address decides price . . . . .                 | 7         |
| 5.4.3    | And the split is geographic . . . . .           | 8         |
| 5.4.4    | Type sets the median, not the listing . . . . . | 9         |
| 5.4.5    | The other findings, briefly . . . . .           | 9         |
| 5.5      | Train / Validation / Test Split . . . . .       | 10        |
| <b>6</b> | <b>Methodology and Training</b>                 | <b>10</b> |
| 6.1      | Models Considered . . . . .                     | 10        |
| 6.2      | Training Setup . . . . .                        | 11        |
| 6.3      | Splitting the Problem . . . . .                 | 11        |
| 6.4      | Hyperparameter Tuning . . . . .                 | 11        |
| 6.5      | Baseline . . . . .                              | 12        |
| <b>7</b> | <b>Evaluation and Results</b>                   | <b>12</b> |
| 7.1      | Evaluation Metrics . . . . .                    | 12        |

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| 7.2       | Results Table . . . . .                                             | 12        |
| 7.2.1     | What the numbers say . . . . .                                      | 13        |
| 7.2.2     | The stricter model kept the accuracy and dropped the size . . . . . | 14        |
| 7.3       | Error Analysis . . . . .                                            | 14        |
| 7.4       | Feature Importance . . . . .                                        | 15        |
| <b>8</b>  | <b>The Meimar Platform</b>                                          | <b>16</b> |
| 8.1       | The map . . . . .                                                   | 16        |
| 8.2       | The estimate, beside the asking price . . . . .                     | 17        |
| 8.3       | Estimating a property that is not listed . . . . .                  | 18        |
| 8.4       | Arabic first . . . . .                                              | 19        |
| 8.5       | Where the work happens . . . . .                                    | 20        |
| <b>9</b>  | <b>Challenges</b>                                                   | <b>20</b> |
| 9.1       | The data was text, not columns . . . . .                            | 21        |
| 9.2       | One model was not enough . . . . .                                  | 21        |
| 9.3       | The best model was memorising . . . . .                             | 21        |
| <b>10</b> | <b>Future Work</b>                                                  | <b>21</b> |
| <b>11</b> | <b>Discussion</b>                                                   | <b>22</b> |
| 11.1      | Interpretation of Results . . . . .                                 | 22        |
| 11.2      | Limitations . . . . .                                               | 22        |
| 11.3      | Ethical and Practical Considerations . . . . .                      | 22        |
| <b>12</b> | <b>References</b>                                                   | <b>23</b> |

# 1 Abstract

---

Saudi property platforms show a price and little else. This project asks a harder question: is that price reasonable? I started from 810,888 public listings, and the first obstacle was that the useful details were not in columns at all — they were buried in free Arabic advertisement text. I recovered them two ways: hand-written normalisation and regular expressions for the easy patterns, and a local Gemma 4 26B model reading roughly 211,000 listings and returning one JSON object per row for the rest. That produced a clean table of 781,398 rows.

I then answered nine business questions about the market on the GPU, and trained eleven models on one fixed split: gradient boosting against linear, ridge and lasso baselines. One model for everything was not good enough, so I split the problem in two — land priced per square meter, buildings priced per unit. That single decision lifted the score on the log target from **0.757** to **0.884** and **0.873**. The final model prices half of all buildings within 10% of the asking price, using 15 times fewer leaves than my first attempt. Every number here is measured on listings the model never saw. The models are not left in a notebook: they run inside **Meimar**, a working map-first platform that carries an estimate for 600,941 listings.

## 2 Introduction

---

Most Saudi real estate platforms stop at the advertisement. They show the price, the specifications, the area, and if you are lucky a link to Google Maps. That is a listing board, and it leaves the buyer to work out the hard part alone.

The hard part is context. Is 3,000,000 SAR a fair price for this villa, in this district, at this size? Which neighbourhoods are expensive, and by how much? What is nearby?

Meimar is my answer. It is not another listing board — it is a platform built on data, analysis and machine learning, designed to give the user information deep enough to make a decision with. This report covers the data science behind it: how the data was recovered, what the market actually looks like, and how well a model can price a property from an advertisement.

## 3 Problem Statement

---

### 3.1 Background and Context

Saudi Arabia does not publish what properties actually sold for. What is public is what sellers *ask*. Millions of these asking prices exist across listing platforms, and nobody turns them into a reference point.

At the same time, the technology to do it is now cheap. Machine learning models, and techniques like retrieval and function calling, can run on ordinary hardware. Some platforms avoid them because of audience and running cost — which is understandable, and also an opening.

### 3.2 Why the Problem Matters

A buyer has no way to check a price. A seller has no way to set one. Both are negotiating in the dark against someone who does this professionally.

An estimate built from hundreds of thousands of comparable listings gives both sides the same starting number. It does not need to be perfect. It needs to be honest about what it is.

### 3.3 Problem Definition

Table 1: The machine learning task

| Item          | Definition                                                                                                                                      |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>  | 38 columns per listing: area, rooms, street width, city, district, property type, and 15 binary features recovered from the advertisement text. |
| <b>Target</b> | Buildings: $\log_{10}(\text{price})$ . Land: $\log_{10}(\text{price per m}^2)$ .                                                                |
| <b>Task</b>   | Supervised regression.                                                                                                                          |
| <b>Output</b> | An expected price, plus the range it plausibly falls in.                                                                                        |

**Given** a property's size, location, type and features, **predict** the price a comparable listing would ask.

## 4 Objectives

1. Build a model that predicts **land** prices per square meter, and a second that predicts **building** prices per unit.
2. Compare gradient boosting against three linear baselines — linear, ridge and lasso — on one identical split.
3. Recover the missing values hidden in Arabic advertisement text, using on-device AI rather than dropping the rows.
4. Answer nine business questions about the Saudi sale market, using GPU dataframes throughout.
5. Identify which features actually drive the price, and report the final models on data they never trained on.
6. Deliver the result as a working platform — not a notebook — where a user can browse the market on a map and read an estimate beside every asking price.

## 5 Data

### 5.1 Data Source

Table 2: Dataset provenance

|                   |                                                                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Source</b>     | Saudi Arabia Real Estate Dataset, Kaggle                                                                                                                      |
| <b>URL</b>        | <a href="https://www.kaggle.com/datasets/mohdph/saudi-arabia-real-estate-dataset">https://www.kaggle.com/datasets/mohdph/saudi-arabia-real-estate-dataset</a> |
| <b>Collection</b> | Public listings scraped from a Saudi property platform                                                                                                        |
| <b>Raw size</b>   | 810,888 listings                                                                                                                                              |
| <b>Nature</b>     | Advertised asking prices — not completed transactions                                                                                                         |

### 5.2 Dataset Description

The raw file is mostly text. Two columns — `title` and `content` — are 87% of the file: 196 MB of advertisement body and 22 MB of headlines. Everything a model needs was written in those two columns as prose.

Table 3: Main features after cleaning

| Feature                          | Data Type   | Description                                          |
|----------------------------------|-------------|------------------------------------------------------|
| <code>area_m2</code>             | Numeric     | Property area in square meters                       |
| <code>bedrooms</code>            | Numeric     | Number of bedrooms                                   |
| <code>living_rooms</code>        | Numeric     | Number of living rooms                               |
| <code>bathrooms</code>           | Numeric     | Number of bathrooms                                  |
| <code>street_width_m</code>      | Numeric     | Width of the facing street                           |
| <code>city</code>                | Categorical | 97 cities                                            |
| <code>district</code>            | Categorical | 1,979 city–district pairs                            |
| <code>estate_type</code>         | Categorical | Villa, apartment, land, building, floor, and 10 more |
| <code>has_ac ... has_yard</code> | Binary      | 15 flags recovered from the text                     |
| <code>*_missing</code>           | Binary      | Marks a value that was absent, not zero              |
| <code>price</code>               | Numeric     | <b>Target</b> — SAR, or SAR/m <sup>2</sup> for land  |

### 5.3 Data Preprocessing

This was the hardest part of the project, so it gets its own figure.

Nulls were everywhere, and dropping them would have thrown away most of the dataset. But the values were not missing — they were written in the advertisement instead of the column. So I recovered them in two passes.

**Pass one, by hand.** Join title and content, normalise Arabic letters and digits, delete negated phrases so that “no air conditioning” never counts as air conditioning, then match patterns.

**Pass two, by model.** Around 211,000 rows still had gaps. Each was sent to a local Gemma 4 26B with a schema, and returned one JSON object. Seven columns were filled this way.

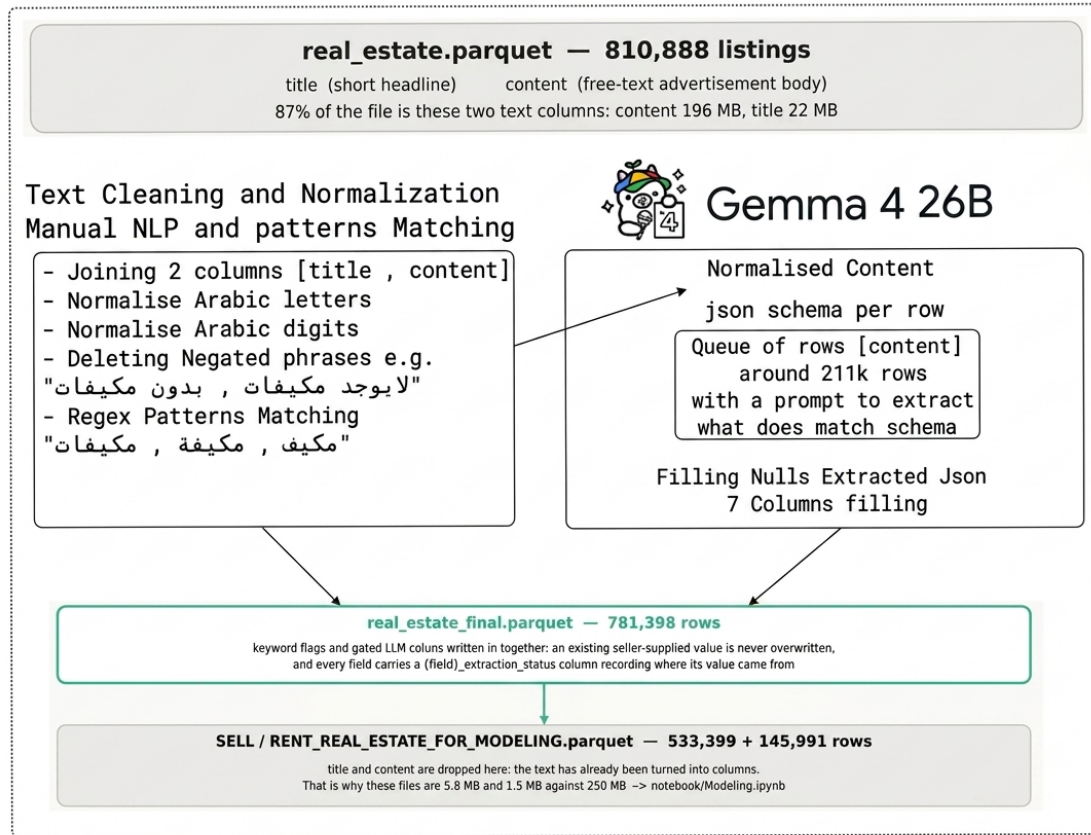


Figure 1: Filling null values by extracting them from the advertisement text. A seller-supplied value is never overwritten, and every field carries a status column recording where its value came from.

The result is `real_estate_final.parquet` with 781,398 rows, which splits into 533,399 sale listings and 145,991 rental listings for modelling. Title and content are dropped at that point — the text has already become columns, which is why the modelling files are 5.8 MB and 1.5 MB against 250 MB.

## 5.4 Exploratory Data Analysis

Nine questions, all answered on the same population: sale listings, price per m<sup>2</sup> between 500 and 60,000 SAR, area between 50 and 10,000 m<sup>2</sup>. Medians, not means — both price and area are skewed hard enough that a mean describes the tail instead of the market. A district needs 100 listings before its median is quoted.

### 5.4.1 The supply is one city

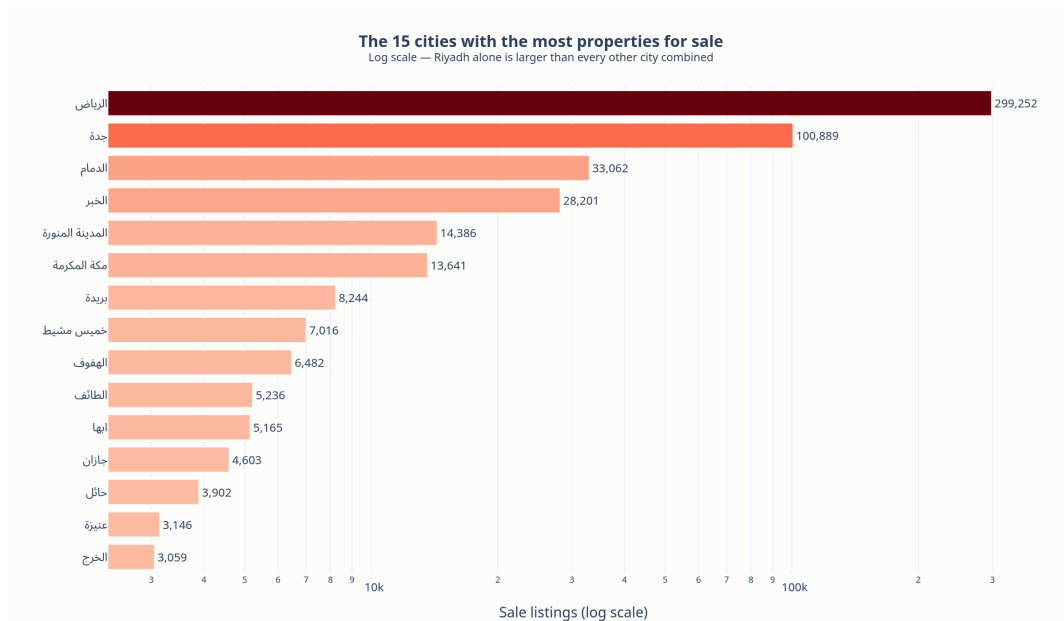


Figure 2: Q4 — sale listings per city. The scale is logarithmic, which is the finding.

Riyadh holds 299,252 sale listings: 53% of the country, nearly 3 times Jeddah. The top three cities are 77% of everything; the remaining 87 cities share under 8%. Any model trained on all the data is really a Riyadh model.

### 5.4.2 Address decides price

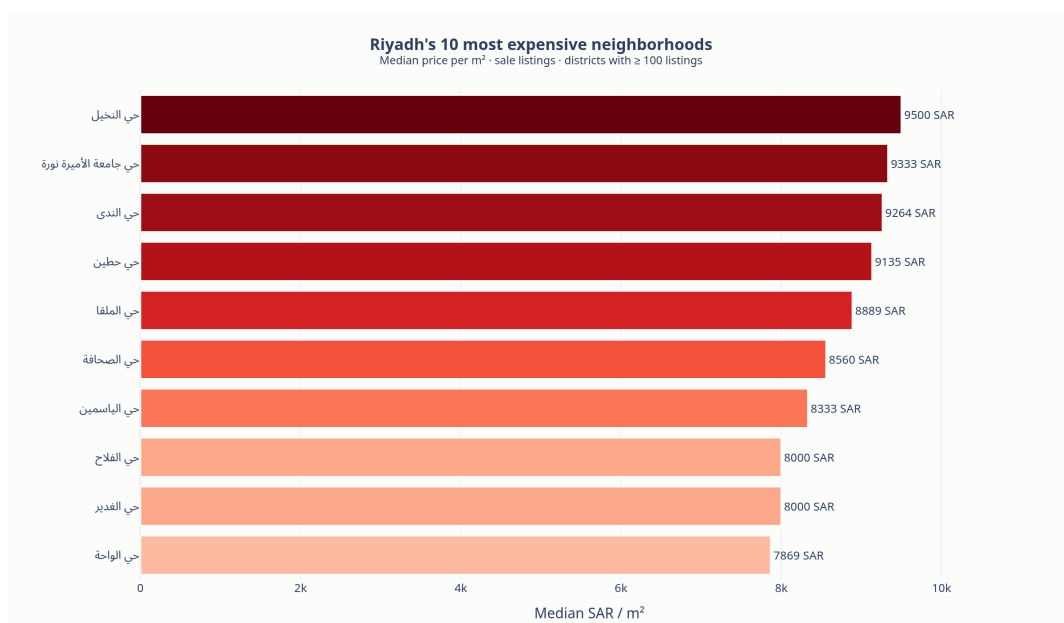


Figure 3: Q1 — Riyadh's most expensive districts by median SAR/m<sup>2</sup>.



Al Nakheel leads at 9,500 SAR/m<sup>2</sup>. The cheapest district, Al Ha'ir, sits at 760 — a **12.5 times** spread inside one city. This is the single strongest argument for keeping `district` in the model.

### 5.4.3 And the split is geographic

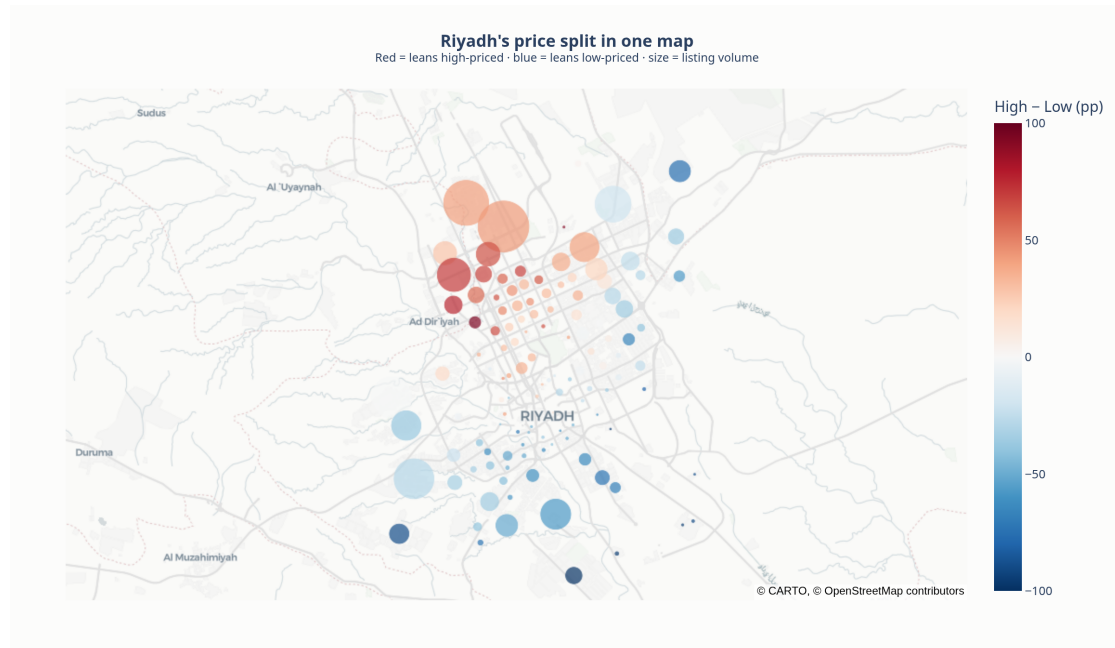


Figure 4: Q8 and Q9 in one map — which side of the market each district leans to. Dark = expensive, light = cheap, size = listing volume.

The expensive districts border each other in the north; the cheap ones spread across the south. Almost no district sits near the middle. Riyadh is not one market with a gradient — it is two markets meeting around the middle ring. That is why `district` behaves like a location feature and not a random label.

#### 5.4.4 Type sets the median, not the listing

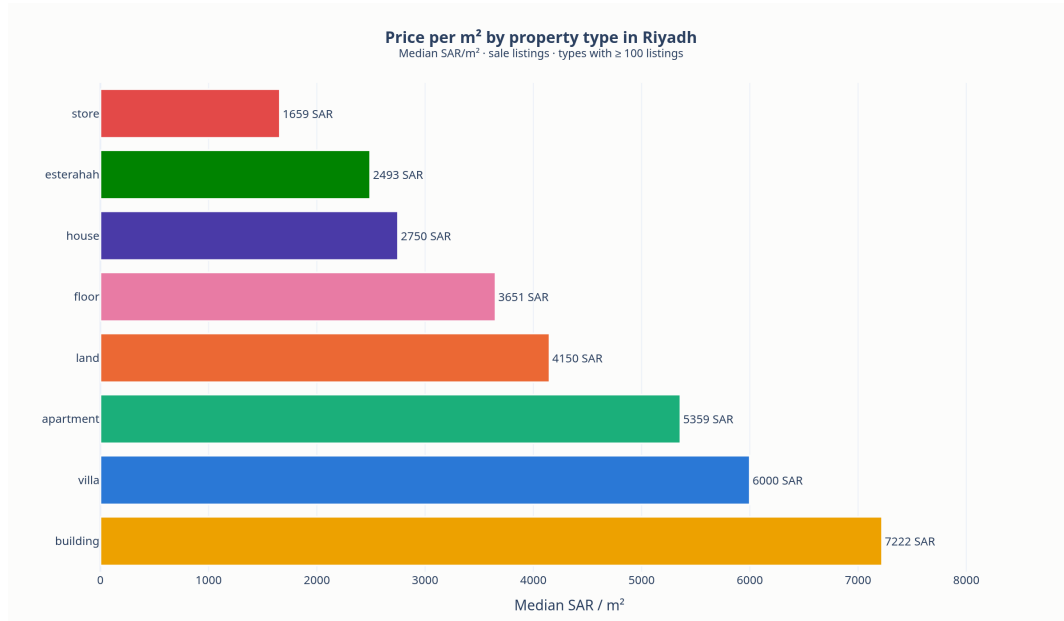


Figure 5: Q7 — median SAR/m<sup>2</sup> by property type in Riyadh.

Buildings top the list because they are priced as income assets, not by area. But the ranges overlap heavily: type separates the medians and not the individual property. Area and location have to do the rest of the work.

#### 5.4.5 The other findings, briefly

Table 4: Remaining EDA questions

| Q         | Finding                                                                                                                                                      |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Q2</b> | The cheap districts are industrial or outer-edge: refineries, new industrial city, Banban. Not housing demand.                                               |
| <b>Q3</b> | No national “typical property” exists. Riyadh is villas (43.4%), Jeddah is apartments (54.2%), Khobar and Madinah are land.                                  |
| <b>Q5</b> | Size follows land availability, not wealth. Hail is largest (500 m <sup>2</sup> ), Jeddah smallest (240 m <sup>2</sup> ).                                    |
| <b>Q6</b> | Area and price per m <sup>2</sup> correlate at only 0.20 across Riyadh districts — they are nearly independent signals, which is exactly what a model wants. |

## 5.5 Train / Validation / Test Split

Table 5: Split configuration

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <b>Ratio</b>  | 70% train / 30% test                                                            |
| <b>Seed</b>   | <code>random_state=42</code>                                                    |
| <b>Shared</b> | The identical split is reused by all eleven models                              |
| <b>Tuning</b> | Hyperparameters searched with K-fold inside the training split only             |
| <b>Why</b>    | The test split is touched once, at the end. Nothing is chosen by looking at it. |

## 6 Methodology and Training

Prices run from a few thousand riyals to hundreds of millions. On that scale a handful of enormous listings would decide the entire error. So every model trains on `log1p(price)`, and every error is converted back into riyals before it is reported.

### 6.1 Models Considered

Table 6: Machine learning models considered

| Model             | Reason for Consideration                                                                       |
|-------------------|------------------------------------------------------------------------------------------------|
| <b>xgb_wide</b>   | Gradient boosting, free search. Handles categories natively and finds interactions on its own. |
| <b>xgb_native</b> | The same, but constrained — to test whether the first model was simply memorising.             |
| <b>Linear</b>     | The honest floor. If boosting cannot beat it, boosting is not worth its cost.                  |
| <b>Ridge</b>      | Linear with a penalty, to test whether the linear failure was overfitting.                     |
| <b>Lasso</b>      | Linear with feature selection, same question from the other side.                              |

Each was trained three times — once on all types together (*universal*), once on buildings, once on land — giving eleven trained models.

## 6.2 Training Setup

Table 7: Experimental setup

|                    |                                                                                |
|--------------------|--------------------------------------------------------------------------------|
| <b>Language</b>    | Python                                                                         |
| <b>Libraries</b>   | XGBoost, cuML, cuDF (RAPIDS), scikit-learn, DuckDB                             |
| <b>Hardware</b>    | NVIDIA RTX 3090, 24 GB                                                         |
| <b>Loss</b>        | Squared error on the log target                                                |
| <b>Selection</b>   | Early stopping on the validation fold                                          |
| <b>Measurement</b> | Time, GPU utilisation and memory polled every 50 ms <i>while</i> the code runs |

I measured the GPU because a model is only worth what it costs. Preparation, search and final training are timed separately, because they are three different costs. The card is told to finish before each timer stops — otherwise I would be timing Python, not the GPU.

## 6.3 Splitting the Problem

One model for every property type was not good enough. Land is about 40% of the listings and its price works differently: it comes from area and location, not from rooms. Mixing it with houses drags both down.

- **Land** — predict price *per square meter*, because a plot's price grows with its area. Eight columns only: a bare plot has no bedrooms, no kitchen and no furniture, so those columns are noise.
- **Buildings** — everything else, predict the price itself.

## 6.4 Hyperparameter Tuning

The first search was free to choose, and it chose deep: depth 12 and 1,414,819 leaves. It scored far better on rows it had seen than on new ones. So I cut the search down.

Table 8: Search constraints for `xgb_native`

|                   |                                                                                                                              |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Method</b>     | Randomised search with K-fold cross-validation                                                                               |
| <b>Depth</b>      | At most 8                                                                                                                    |
| <b>Leaf size</b>  | At least 10 listings behind every leaf, so no leaf fits one house                                                            |
| <b>Penalty</b>    | <code>reg_lambda</code> of 5 or more                                                                                         |
| <b>Categories</b> | <code>city</code> and <code>district</code> kept as categories, so XGBoost splits on them instead of reading them as numbers |
| <b>Scoring</b>    | Each candidate scored once, after training                                                                                   |

## 6.5 Baseline

Linear, ridge and lasso. They cannot read categories and they care about scale, so they get their own preparation: city and district encoded as numbers, constant columns dropped, every feature scaled — fitted on the training split alone.

## 7 Evaluation and Results

### 7.1 Evaluation Metrics

Table 9: Evaluation metrics

| Metric            | Purpose                                                                                                            |
|-------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>R2_log</b>     | Fit on $\log_{10}(\text{price})$ — the target the model actually trains on.                                        |
| <b>R2</b>         | The same idea, in money.                                                                                           |
| <b>MAE</b>        | Average error, in money.                                                                                           |
| <b>RMSE</b>       | Error again, but large misses count much more.                                                                     |
| <b>within_10%</b> | Share of listings priced within 10% of the truth. The easiest number to read, and the one a user would care about. |

Money means SAR, except for land, where it is SAR per square meter — because that is what the land model predicts.

### 7.2 Results Table

Table 10: Accuracy on the test split. Errors are in the unit each model produces.

| Target    | Model             | R2_log | R2     | MAE       | within_10% | Unit               |
|-----------|-------------------|--------|--------|-----------|------------|--------------------|
| universal | <b>xgb_wide</b>   | 0.757  | 0.165  | 1,853,984 | 46.9       | SAR                |
| buildings | <b>xgb_wide</b>   | 0.887  | 0.769  | 367,645   | 56.4       | SAR                |
| buildings | <b>xgb_native</b> | 0.884  | 0.771  | 397,582   | 50.4       | SAR                |
| buildings | linear            | 0.543  | 0.233  | 954,814   | 16.6       | SAR                |
| buildings | ridge             | 0.546  | 0.253  | 950,998   | 16.9       | SAR                |
| buildings | lasso             | 0.546  | 0.252  | 950,977   | 16.8       | SAR                |
| land      | <b>xgb_wide</b>   | 0.875  | 0.845  | 456       | 46.2       | SAR/m <sup>2</sup> |
| land      | <b>xgb_native</b> | 0.873  | 0.830  | 510       | 40.7       | SAR/m <sup>2</sup> |
| land      | linear            | 0.126  | -1.876 | 1,845     | 6.9        | SAR/m <sup>2</sup> |
| land      | ridge             | 0.126  | -1.875 | 1,845     | 6.9        | SAR/m <sup>2</sup> |
| land      | lasso             | 0.126  | -1.674 | 1,845     | 6.9        | SAR/m <sup>2</sup> |

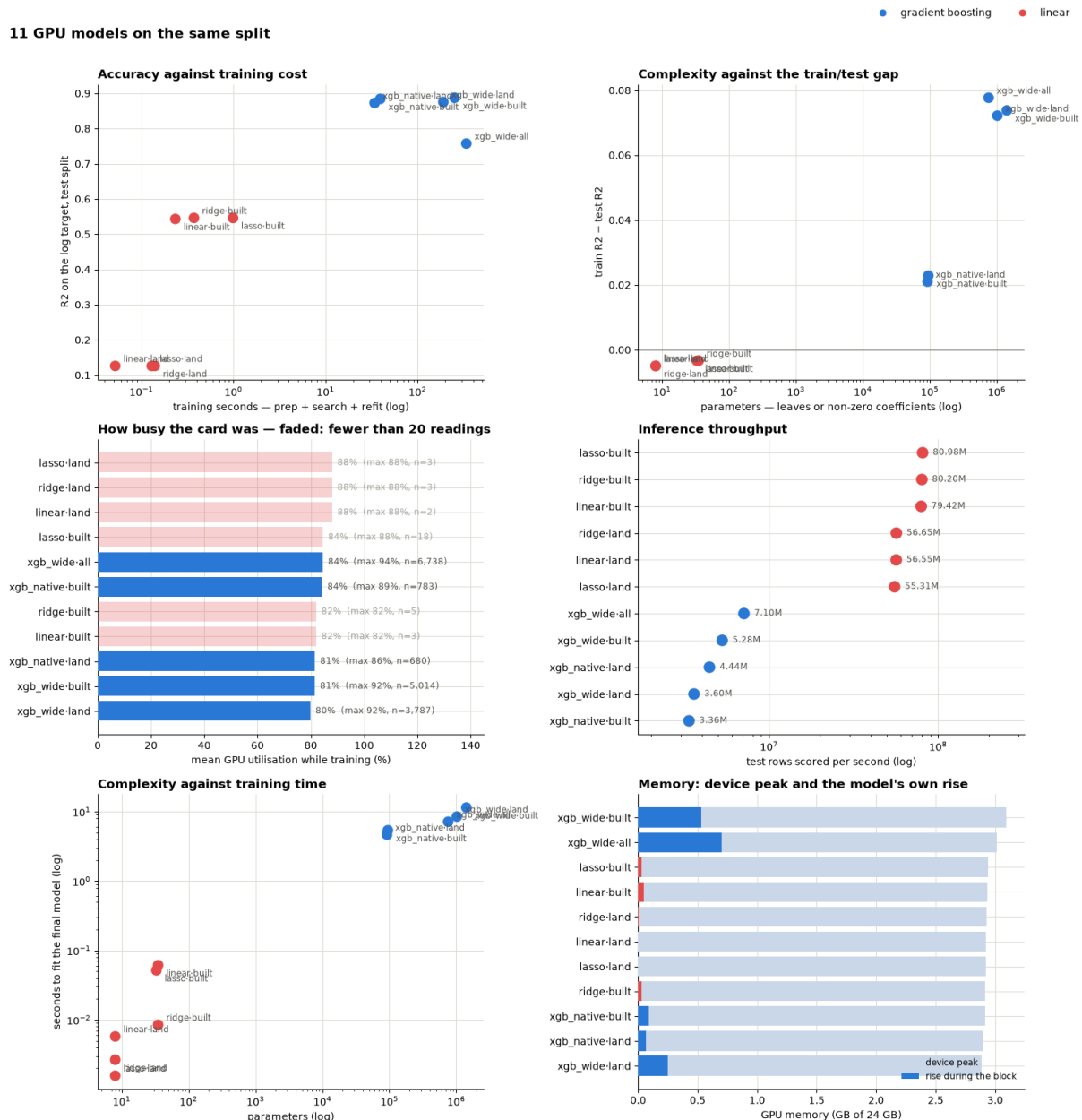


Figure 6: All eleven models on the same split — what each one bought, and what it cost.

### 7.2.1 What the numbers say

- **Boosting wins.** It prices 50.4% of buildings and 40.7% of land within 10% of the truth. The linear models manage 16.9% and 6.9%.
- **Splitting the data helped most.** One universal model reaches  $R2_{\log}$  0.757; separate models reach 0.884 and 0.873.
- **Linear models fail on land because of the codes.** District is just a number to them, so district 1235 looks one bigger than 1234. Location is the whole price of a plot, so they cannot learn it — 0.126 against 0.873.
- **A penalty does not save them.** Ridge scores the same from alpha 0.001 to 10. The

problem is too little learning, not too much.

- **Read land in SAR per m<sup>2</sup>.** In total riyals R2 falls to 0.187, because a few enormous plots then own the error.

### 7.2.2 The stricter model kept the accuracy and dropped the size

Table 11: Buildings — first attempt against the constrained one

|                     | xgb_wide  | xgb_native   |
|---------------------|-----------|--------------|
| Leaves              | 1,414,819 | 95,027       |
| R2_log              | 0.887     | 0.884        |
| R2                  | 0.769     | 0.771        |
| within_10%          | 56.4      | 50.4         |
| Train – test R2_log | 0.074     | <b>0.023</b> |
| Fit time (s)        | 11.5      | <b>5.4</b>   |

**Best model: xgb\_native**

15 times smaller, the same test score, and the train–test gap falls from 0.074 to 0.023. It gives up 6 points of within\_10% and buys back a model that is not memorising. That is the one I would deploy.

## 7.3 Error Analysis

The model fails in three places, and all three are the data rather than the algorithm.

Table 12: Where the model struggles

| Case                     | Why                                                                                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Very large plots</b>  | A few enormous listings dominate the squared error. In total riyals land R2 falls from 0.845 to 0.187 — which is why land is priced per m <sup>2</sup> . |
| <b>Thin districts</b>    | A district with 145 listings gets a median as confidently as one with 12,609. The model has little to learn from.                                        |
| <b>Small cities</b>      | 87 cities share under 8% of listings. They never get enough weight to be learned properly.                                                               |
| <b>Overlapping types</b> | Villa and land have wide, overlapping price ranges. Type alone cannot separate individual listings.                                                      |

## 7.4 Feature Importance

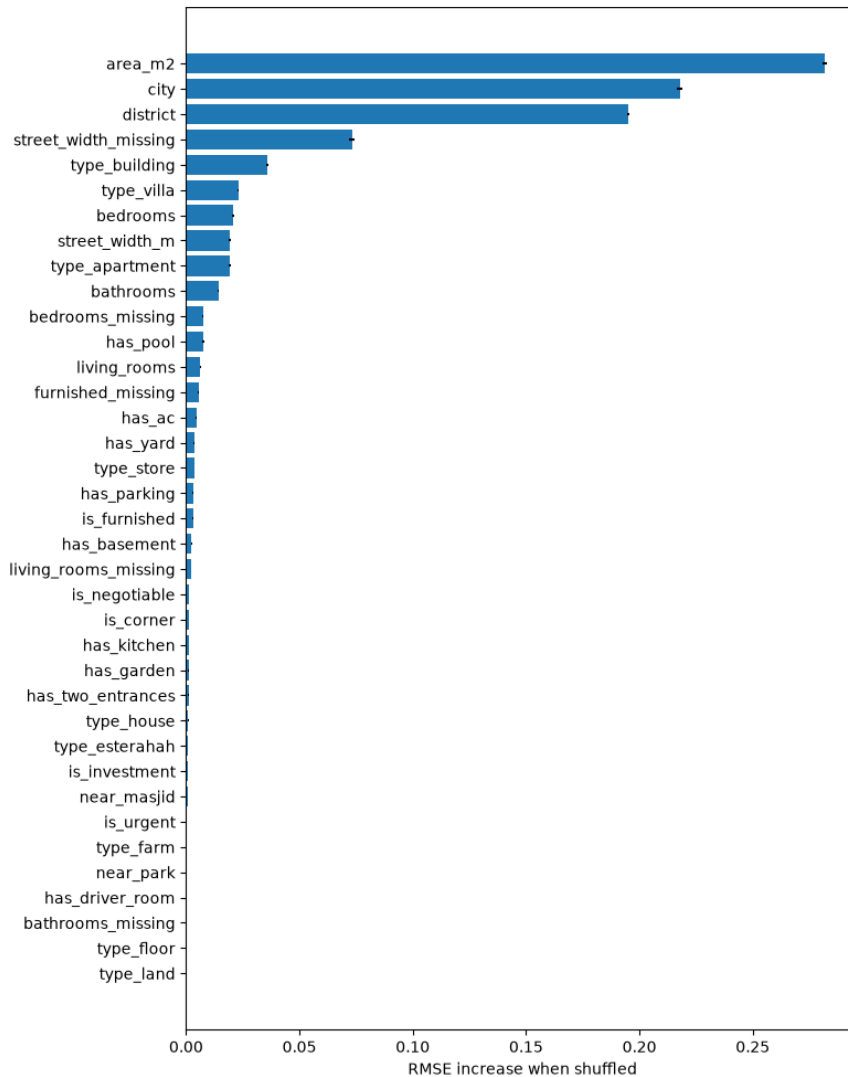


Figure 7: Permutation importance for the buildings model — how much RMSE rises when a column is shuffled.

Three features carry the model: **area**, **city** and **district**. Everything else is a rounding error beside them.

This confirms the EDA. Q1 and Q2 found a 12.5 times price spread between Riyadh districts; Q6 found that area and location are nearly independent signals. The model agrees — it spends almost all of its attention on size and address.



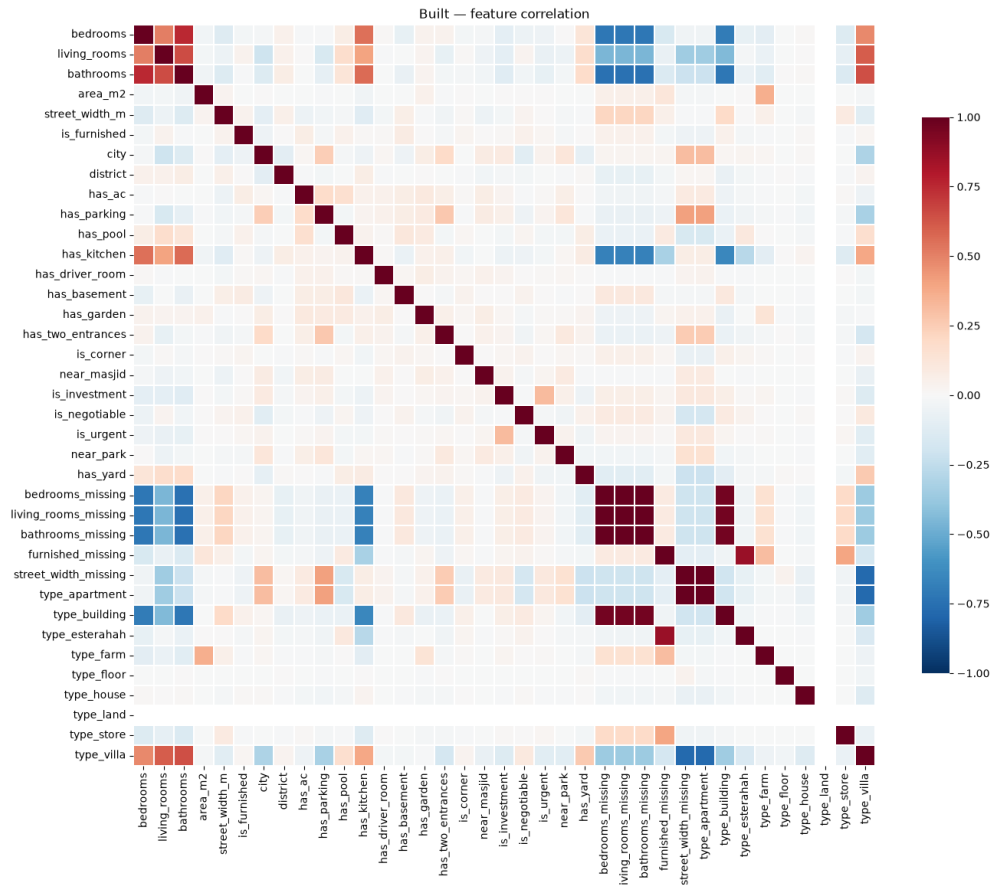


Figure 8: Correlation between the building features. Two features saying the same thing would make the model bigger for nothing.

## 8 The Meimar Platform

A model that lives in a notebook helps nobody. So the two models were scored over the whole dataset and built into a platform: **781,382** listings on one map, **600,941** of them carrying a price estimate.

### 8.1 The map

Zoomed out, the country is a density surface. At street level every property becomes a pin with its price on it.



Figure 9: Riyadh at street level. Each pin is one listing, labelled with its asking price.

## 8.2 The estimate, beside the asking price

Clicking a pin is where the model appears. The panel shows the asking price, the model's estimate, how far apart they are, and the range the estimate plausibly falls in — next to the services measured around that point.



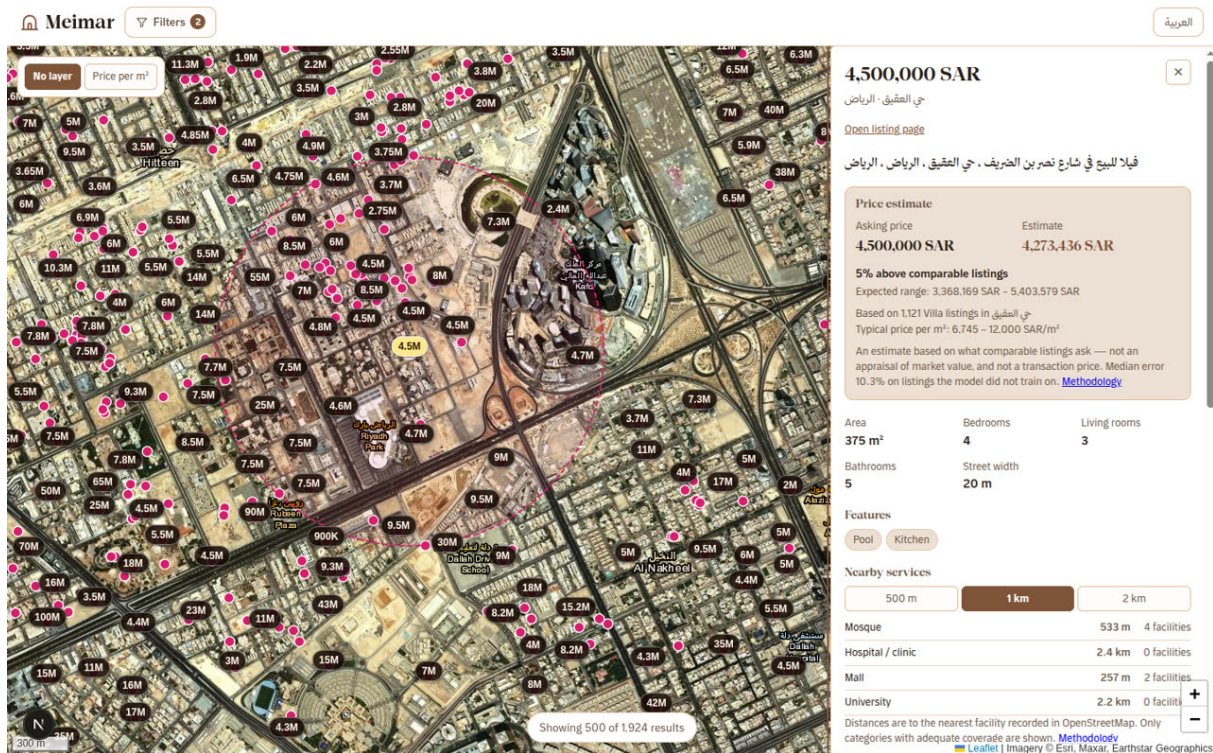


Figure 10: A villa asking 4,500,000 SAR against an estimate of 4,273,436 — 5% above comparable listings. The dotted circle is the selected radius, and the distances beside it come from OpenStreetMap.

**Every stored estimate is out-of-fold.** A model that trained on a listing predicts it too well, which would make most listings look fairly priced purely because they were training rows. Each listing is scored by a fold that never saw it.

### 8.3 Estimating a property that is not listed

The same models answer for a property that does not exist yet — describe it, and get a number with its range.

Figure 11: A 400 m<sup>2</sup> villa in Al Malqa, Riyadh, estimated at 2,319,063 SAR with an expected range beside it.

This one path needs a server. The boosters split on `city` and `district` as categories, and browser inference formats express only numeric comparisons, so these models cannot be converted to run in the page. Stored listings do not care — their features never change, so their estimates are computed once, in advance.

## 8.4 Arabic first

The platform is fully bilingual, right-to-left included.



Figure 12: The same panel in Arabic.

## 8.5 Where the work happens

The scalable part is that there is almost no server. There is no listings API and no database behind the map. The browser downloads compressed columnar files and queries them itself.

Table 13: What runs where

| Component            | Runs        | Purpose                                                                       |
|----------------------|-------------|-------------------------------------------------------------------------------|
| <b>DuckDB-WASM</b>   | Browser     | Filters and aggregates 781,382 listings client-side                           |
| <b>Parquet files</b> | Static      | Read by HTTP range request — about 1.5 MB per click instead of the whole file |
| <b>Leaflet</b>       | Browser     | Map, from country view to street level                                        |
| <b>XGBoost</b>       | Precomputed | Out-of-fold estimates stored per listing                                      |
| <b>FastAPI</b>       | Server      | The one live path: free-form estimates                                        |

Pushing the query engine to the client is what makes 781,382 rows feel instant without paying for a database that answers every pan and zoom.

## 9 Challenges

## 9.1 The data was text, not columns

**What happened.** Most useful fields were null. The information existed, but as Arabic prose inside the advertisement.

**What I tried.** Dropping the rows — which would have destroyed the dataset. Then regular expressions alone, which handled clean phrasing and missed everything written loosely.

**What I chose.** Both, in order. Deterministic cleaning first, then Gemma 4 26B on the remaining 211,000 rows with a strict JSON schema, and a status column recording where every value came from. A seller-supplied value is never overwritten.

**Outcome.** Seven columns filled and 781,398 usable rows kept.

## 9.2 One model was not enough

**What happened.** A single model over all property types reached  $R^2_{\log}$  0.757 and  $R^2$  0.165 — close to useless in money terms.

**What I tried.** More trees and a longer search. It barely moved.

**What I chose.** Split the problem: land per square meter, buildings per unit, each with its own feature list.

**Outcome.** 0.884 and 0.873. The best single decision in the project.

## 9.3 The best model was memorising

**What happened.** The free search chose depth 12 and 1.4 million leaves, with a 0.074 train–test gap.

**What I tried.** Accepting it — it did have the highest score.

**What I chose.** Constrain the search: depth 8, minimum leaf size, a penalty, and one honest scoring pass per candidate.

**Outcome.** 15 times smaller, the same test score, and a third of the gap.

## 10 Future Work

---

1. **Add location intelligence to the model.** Distance to the nearest mosque, hospital, mall and university is already computed for the platform. Feeding it back into training should help most where the district is thin.
2. **A conversational interface with function calling.** Let a user ask in Saudi dialect and have the model call the platform’s own functions — district statistics, price estimate, comparable listings — so it retrieves real numbers instead of generating them.
3. **Weight the small cities.** 87 cities share under 8% of the data. A per-region model, or



sample weighting, would stop them being ignored.

4. **Track listings over time.** The dataset is one snapshot. Repeated observations would turn asking prices into a trend, and a trend is far more useful than a level.

## 11 Discussion

---

### 11.1 Interpretation of Results

All six objectives were met. Both models exist and both are measured on unseen data. Five algorithms were compared on one split. The nulls were recovered rather than dropped. Nine questions were answered on the GPU. The important features are identified and agree with the exploration. And the result is a platform a person can actually open, not a notebook.

The deeper result is that **the split mattered more than the algorithm**. Moving from one model to two lifted  $R^2_{\log}$  by 0.127. Every tuning decision after that moved it by hundredths. Understanding that land and buildings are different problems was worth more than any hyperparameter.

### 11.2 Limitations

- **These are asking prices, not sale prices.** The model answers “what do comparable listings ask?” — never “what is this worth?”. It cannot be used as a valuation.
- **It is mostly a Riyadh model.** 53% of the data is one city. Expect it to be weakest exactly where the data is thinnest.
- **One snapshot in time.** There is no trend, no seasonality, and no way to know if the market has moved since.
- **Extracted values are inferred.** Where a value came from the text rather than the seller, it is a reading of an advertisement written to sell.

### 11.3 Ethical and Practical Considerations

The dataset is public listings, and the platform is built to comply with the Personal Data Protection Law (PDPL) — no personal identifiers are kept, and nothing identifies a seller.

The cost of a wrong prediction is not symmetric. Overestimating helps a seller and hurts a buyer; underestimating does the reverse. That is why the platform shows an expected *range* beside every estimate and states plainly, at every number, that these are asking prices. A model that looks authoritative is more dangerous than one that shows its uncertainty.

## 12 References

---

1. Saudi Arabia Real Estate Dataset. Kaggle. Retrieved from <https://www.kaggle.com/datasets/mohdph/saudi-arabia-real-estate-dataset> on 13/08/2026.
2. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
3. RAPIDS Development Team. *RAPIDS: GPU-Accelerated Data Science* (cuDF, cuML). Documentation. <https://rapids.ai>
4. Google DeepMind. *Gemma*. Model documentation. <https://ai.google.dev/gemma>
5. Ollama. *Local large language model runtime*. Documentation. <https://ollama.com>
6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
7. Raasveldt, M., & Mühleisen, H. (2019). *DuckDB: An Embeddable Analytical Database*. Proceedings of the 2019 International Conference on Management of Data (SIGMOD), 1981–1984.
8. DuckDB Labs. *DuckDB-WASM* (version 1.33.1). In-browser analytical database. <https://github.com/duckdb/duckdb-wasm>
9. Apache Software Foundation. *Apache Parquet* and *Apache Arrow* (version 17.0). Columnar storage and in-memory format. <https://parquet.apache.org>
10. Vercel. *Next.js* (version 16.3) and *React* (version 19.2). Web application framework. <https://nextjs.org>
11. Agafonkin, V. *Leaflet* (version 1.9.4). JavaScript library for interactive maps. <https://leafletjs.com>
12. OpenStreetMap contributors. *OpenStreetMap*. Geographic data used for facility distances, licensed under ODbL. <https://www.openstreetmap.org>
13. Ramírez, S. *FastAPI* (version 0.115) and *Pydantic* (version 2.7). Python web framework and data validation. <https://fastapi.tiangolo.com>